1. You are Codex operating as an agent inside this repo. Your job is to build an App Store–ready MVP for the “Zenith Legal Candidate Portal” described in the spec files in /mnt/data:
2.
3. - /mnt/data/email-app-requirements.md (primary MVP requirements)
4. - /mnt/data/zl-app-transcropt.md (product intent, messaging, alerts, workflow)
5. - /mnt/data/firm-list-2026.md (canonical firm list; must be imported as master data)
6. - /mnt/data/PROJECT_LOG.md (notes/history; treat as supporting info)
7.
8. GOAL
9. Ship an iOS App Store–ready MVP (and Android-capable) with:
10. 1) Candidate mobile app (Expo React Native + TypeScript)
11. 2) Admin/Recruiter web console (Next.js + TypeScript)
12. 3) Managed backend (Firebase preferred: Auth + Firestore + Cloud Functions if needed)
13. 4) Push notifications and real-time messaging
14. 5) Role-based access control (candidate vs admin/team)
15.
16. NON-NEGOTIABLE MVP REQUIREMENTS (from the spec files)
17. - No traditional password signup. Candidate onboarding collects name, email, mobile; uses OTP verification.
18. - Candidate sets preferred cities and practice area.
19. - Messaging: iMessage-style thread between candidate and Zenith team; history persists; push on new message.
20. - Firm status dashboard: candidate only sees firms in allowed statuses and only those related to them.
21. Allowed statuses visible to candidates:
22. - authorization pending
23. - submitted waiting
24. - interview
25. - rejected
26. - offer

- Candidate can approve/decline authorization requests. Candidate cannot otherwise edit firm statuses.
- Recruiter (Mason Kalfus) can add firms to a candidate, request authorization, and update statuses.
- Calendar/appointments: create/update/cancel with reminders/alerts.
- Prominent “tap to call / tap to email Mason” on candidate home screen.
- Add “Delete my account/data” flow (required for App Store if accounts exist).
- Import /mnt/data/firm-list-2026.md into the backend as the canonical Firms dataset.

STACK (use exactly this unless impossible)

- Mobile: Expo (React Native) + TypeScript
- Web admin: Next.js + TypeScript
- Backend: Firebase Auth + Firestore + Firebase Cloud Functions (only if needed)
- Notifications: Expo Notifications (or FCM/APNs via Firebase where appropriate)
- UI approach: simple, clean, minimal; use React Navigation on mobile.

DELIVERABLES

A) Working code for mobile app + admin web app

B) Firestore data model + security rules enforcing RBAC

C) Seed script to import firm-list-2026.md into Firestore

D) Setup docs: how to run locally, env vars, how to deploy, how to build with EAS for App Store

E) App Store readiness checklist: privacy policy placeholders, data deletion path, permissions rationale

WORK METHOD (IMPORTANT)

- Work in small, reviewable steps. After each step, summarize what changed and what to test.
- Prefer correctness and clear structure over fancy UI.
- When you need decisions, choose sensible defaults and document them.
- Add basic error handling and loading states everywhere.

BUILD PLAN (execute in this exact order)

1) Read and summarize the four spec files. Extract requirements into a checklist.

2) Create a monorepo structure:

/apps/mobile (Expo)

/apps/admin (Next.js)

/packages/shared (types, validation, constants)

3) Define Firestore schema + TypeScript types:

- users (candidate records; include role, preferences)

- firms (seeded canonical firms)

- candidateFirmStatuses (candidateId, firmId, status, timestamps, history)

- conversations/messages (candidateId, senderRole, text, attachments, timestamps)

- appointments (candidateId, time, location/link, notes, status)

- authorizationRequests (candidateId, firmId, state)

4) Implement Firebase Auth:

- OTP-based sign-in (phone or email OTP depending on easiest reliable path)

- Create user profile on first login

5) Implement candidate mobile app screens (React Navigation tabs):

- Home (Mason contact CTA + “Message Zenith”)

- Status (firm list filtered to allowed statuses)

- Calendar (appointments list + detail)

- Profile (preferences + delete account/data)

6) Implement messaging on mobile:

- Real-time message list

- Send message

- Push notification on new message

7) Implement authorization request UX on mobile:

- Show pending authorizations and allow approve/decline

8) Implement admin web console:

- Admin login (role=admin)

- Candidate list + search

- Candidate detail: profile, firms panel (add firm from master list), request authorization, update status
- Messages panel: read/respond
- Appointments panel: create/update/cancel

9) Implement RBAC security rules:

- Candidates can read/write only their own messages, appointments, profile, and authorization responses
- Candidates can read firms and their own candidateFirmStatuses
- Admins can read/write everything relevant

10) Implement firm list import:

- Parse /mnt/data/firm-list-2026.md and seed Firestore firms collection (id, name)

11) Implement account/data deletion:

- Candidate can request deletion; deletes Firestore user doc + related subcollections, and disables auth user if possible

12) Add docs:

- README with setup
- ENV variable templates
- Firebase project setup
- EAS build + submit steps

13) Add a minimal QA checklist and ensure app runs end-to-end locally.

ACCEPTANCE TESTS (must pass)

- Candidate can onboard via OTP and set preferences.
- Candidate can message Zenith; admin sees and replies; candidate receives reply and push.
- Admin can add firm to candidate and request authorization; candidate approves; admin updates status to submitted/interview/etc; candidate Status screen updates in real time.
- Candidate cannot edit statuses except authorization approve/decline.
- Candidate calendar shows appointments created by admin; reminders/alerts work.
- Account deletion removes candidate data.

110. NOW START.

111. First: read the spec files from /mnt/data, produce a requirement checklist, then scaffold the monorepo and commit the baseline structure.

Worked for 3m 55s

Zenith Legal MVP Plan (Expo + Next.js + Firebase)

Summary

We will build a monorepo MVP with a candidate mobile app, admin web console, Firebase backend, real-time messaging, push notifications, RBAC, and firm master-data import.

No code is changed yet (plan mode only).

I will use the spec files found in this repo root (the `/mnt/data` path is not mounted in this environment).

Implementation Sequence (Exact Order)

1. Read/spec checklist: Create one requirements checklist doc from email-app-requirements.md, zl-app-transcropt.md, firm-list-2026.md, PROJECT_LOG.md, and mark each item as `MVP`, `Later`, or `Out`.
2. Monorepo scaffold: Create `/apps/mobile` (Expo + TS + React Navigation), `/apps/admin` (Next.js + TS), `/packages/shared` (types/constants/zod validation), plus root workspaces config and shared scripts.
3. Firestore schema/types: Define collections and strict TypeScript/Zod models for `users`, `firms`, `candidateFirmStatuses`, `conversations/messages`, `appointments`, `authorizationRequests`, `deletionRequests` (for audit), plus Firebase indexes file.
4. Auth setup: Implement candidate auth with two passwordless options (`phone OTP` or `email link OTP-style flow`), where either verified method is allowed; collect name/email/mobile and create profile on first login. Implement admin Google sign-in with allowlist and admin role claim.

5. Mobile core screens: Build React Navigation tabs `Home`, `Status`, `Calendar`, `Profile` with loading/error states and empty states.
6. Messaging: Build per-candidate thread with Firestore realtime listeners, send message, Firebase Storage file upload (any type, max 25MB), and push notifications to candidate on new admin message.
7. Authorization UX: Show pending authorization requests in mobile app and allow candidate approve/decline only.
8. Admin console: Implement admin login, candidate list/search, candidate detail page with profile, firms panel (add from canonical list), request authorization, status updates, messages panel, and appointments panel.
9. RBAC rules: Add Firestore/Storage security rules so candidates can only access their own records and only allowed writes; admins can manage all relevant records.
10. Firm import: Add seed script to parse firm-list-2026.md and import firms into Firestore; skip the `Mc` row by rule.
11. Account/data deletion: Add in-app "Delete my account/data" flow with hard delete of candidate-owned records and auth user removal via Cloud Function, plus deletion audit entry.
12. Docs/setup: Add README, `.env.example`, Firebase setup docs, local emulator docs, EAS build/submit docs, and App Store readiness checklist (privacy policy placeholder + permissions rationale + deletion path).
13. QA/end-to-end: Run emulator-based and local manual acceptance tests across mobile + admin and document pass/fail checklist.

Public APIs / Interfaces / Types

1. `UserRole = 'candidate' | 'admin'`.
2. `CandidateFirmStatus = 'authorization_pending' | 'submitted_waiting' | 'interview' | 'rejected' | 'offer'`.
3. `AuthorizationState = 'pending' | 'approved' | 'declined'`.
4. `AppointmentStatus = 'scheduled' | 'canceled' | 'completed'`.
5. `users/{uid}` fields include profile, role, preferences, verification flags, and push token.

6. `candidateFirmStatuses/{id}` fields include `candidateId`, `firmId`, `status`, `updatedBy`, `history[]`.
7. `conversations/{candidateId}` + `messages/{messageId}` with sender role, text, attachment metadata, timestamp.
8. `appointments/{id}` includes `candidateId`, creator role, schedule metadata, status, reminder flags.
9. `authorizationRequests/{id}` includes `candidateId`, `firmId`, `state`, and timestamps.
10. Cloud Functions endpoints/triggers: `sendPushOnMessageCreate`, `sendPushOnAppointmentChange`, `deleteCandidateAccountData`.

Test Cases and Scenarios

1. Candidate can sign in with phone OTP and with email-link OTP flow; first login creates profile.
2. Candidate sets preferred cities and practice area and can update later.
3. Admin can add firm to candidate from seeded firm list and create authorization request.
4. Candidate can approve/decline authorization request; cannot directly edit firm status.
5. Admin can update firm status; candidate status screen updates in realtime and only shows allowed statuses.
6. Candidate and admin can exchange messages; file attachment upload works up to 25MB; candidate receives push notification.
7. Admin and candidate can create/update/cancel appointments; both see realtime updates; candidate receives reminder/alert.
8. Candidate can only read/write own permitted records (rules test with Firebase Emulator).
9. Admin allowlist enforcement blocks non-admin users from admin console data.
10. Candidate deletion flow removes Auth user and related Firestore/Storage data and leaves audit trace.

Assumptions and Defaults

1. Workspace source files in repo root are treated as canonical because `/mnt/data` is unavailable here.
2. Package manager default will be npm workspaces to keep local setup simple.

3. Admin auth uses Google sign-in + email allowlist + admin custom claim.
4. Candidate auth allows either verified phone or verified email-link method.
5. Messaging supports attachments of any file type up to 25MB (no malware scan in MVP).
6. Push notifications are candidate-only for MVP; admin uses realtime web updates.
7. Appointment edits are allowed for both admin and candidate per your choice.
8. Seed importer skips `Mc` and logs it as skipped for traceability.
9. After each implementation step, we will append bullet updates to project `.md` logs/plans (PROJECT_LOG.md, requirements checklist doc, and implementation plan doc) with "what changed" and "what to test".

Implement plan

Worked for 16m 32s

Implemented. I built the MVP baseline across mobile, admin, shared package, Firebase backend, seed tooling, docs, and QA artifacts.

Step-by-step log (changes + what to test)

- Full 13-step summary is here: implementation_step_log.md
- Requirements checklist is here: requirements_checklist.md

What I added

1. Monorepo + workspaces

   package.json, tsconfig.base.json, .gitignore
2. Shared types/constants/schemas

   packages/shared/src/types/domain.ts, packages/shared/src/constants/options.ts, packages/shared/src/schemas/index.ts
3. Candidate mobile app (Expo + TS)

   apps/mobile/App.tsx, apps/mobile/src/navigation/RootNavigator.tsx, apps/mobile/src/screens/AuthScreen.tsx, apps/mobile/src/screens/StatusScreen.tsx, apps/mobile/src/screens/MessagesScreen.tsx, apps/mobile/src/screens/ProfileScreen.tsx

4. Admin web console (Next.js + TS)
   apps/admin/app/auth/page.tsx, apps/admin/app/dashboard/page.tsx,
   apps/admin/app/dashboard/candidates/[candidateId]/page.tsx
5. Firebase config + RBAC rules + indexes
   firebase.json, firestore.rules, storage.rules, firestore.indexes.json
6. Cloud Functions (push, deletion, admin role helper)
   functions/src/index.ts, functions/src/triggers/messagePush.ts,
   functions/src/triggers/appointmentPush.ts, functions/src/callable/deleteAccount.ts,
   functions/src/callable/adminRole.ts
7. Firm seed import (skips `Mc`)
   functions/src/scripts/seedFirms.ts
8. Setup/deploy/compliance docs
   README.md, .env.example, docs/firebase-setup.md, docs/eas-build.md,
   docs/app-store-readiness.md
9. QA docs
   qa/acceptance-checklist.md, qa/test-run-2026-02-25.md
10. Project markdown logs updated
    PROJECT_LOG.md, general_project_plan.md

Verification I ran

1. `npm run typecheck` -> pass
2. `npm run build --workspace @zenith/shared` -> pass
3. `npm run build --workspace @zenith/functions` -> pass
4. `npm run build --workspace @zenith/admin` -> pass

Mobile runtime + emulator end-to-end tests are documented but not executed yet (needs your Firebase env/project).

Next options

1. I can run Firebase emulators and execute the full acceptance checklist with you.
2. I can set up a first real Firebase project config and wire admin claim bootstrap end-to-end.

3. I can prepare a first TestFlight-ready EAS build profile using your real bundle identifiers and secrets.

+149-27

```
apps/admin/app/dashboard/candidates/[candidateId]/page.tsx
apps/admin/app/dashboard/page.tsx
apps/admin/src/lib/auth.ts
apps/admin/src/lib/firebase.ts
apps/admin/tsconfig.json
apps/mobile/src/screens/HomeScreen.tsx
apps/mobile/src/screens/ProfileScreen.tsx
apps/mobile/src/screens/ProfileSetupScreen.tsx
apps/mobile/src/screens/StatusScreen.tsx
apps/mobile/tsconfig.json
firm-list-2026.md
packages/shared/src/schemas/index.ts
```

112. whats next

Best next step is to wire real Firebase config and run full flow tests.

1. Set env values
   - Copy: cp .env.example .env
   - Fill Firebase keys + Mason contact + admin allowlist.
   - Guide: firebase-setup.md
2. Start local backend + apps
   - `firebase emulators:start`
   - `npm run dev:admin`

- `npm run dev:mobile`

3. Seed canonical firms
    - `npm run seed:firms`
    - Confirm firms imported (and `Mc` skipped).
4. Bootstrap Mason as admin
    - Set custom claim `role=admin` once (initial bootstrap), then use admin console login.
5. Run acceptance checklist
    - Use: acceptance-checklist.md
6. Build for App Store readiness
    - Follow: eas-build.md
    - Then finish: app-store-readiness.md

If you want, I can do step 1 and 2 with you now, command by command.

113. You are Codex operating as an agent inside this repo. Your job is to build an App Store–ready MVP for the “Zenith Legal Candidate Portal” described in the spec files in /mnt/data:
114.
115. - /mnt/data/email-app-requirements.md (primary MVP requirements)
116. - /mnt/data/zl-app-transcropt.md (product intent, messaging, alerts, workflow)
117. - /mnt/data/firm-list-2026.md (canonical firm list; must be imported as master data)
118. - /mnt/data/PROJECT_LOG.md (notes/history; treat as supporting info)
119.
120. GOAL
121. Ship an iOS App Store–ready MVP (and Android-capable) with:
122. 1) Candidate mobile app (Expo React Native + TypeScript)
123. 2) Admin/Recruiter web console (Next.js + TypeScript)
124. 3) Managed backend (Firebase preferred: Auth + Firestore + Cloud Functions if needed)
125. 4) Push notifications and real-time messaging
126. 5) Role-based access control (candidate vs admin/team)
127.

NON-NEGOTIABLE MVP REQUIREMENTS (from the spec files)

- No traditional password signup. Candidate onboarding collects name, email, mobile; uses OTP verification.
- Candidate sets preferred cities and practice area.
- Messaging: iMessage-style thread between candidate and Zenith team; history persists; push on new message.
- Firm status dashboard: candidate only sees firms in allowed statuses and only those related to them.

Allowed statuses visible to candidates:

- authorization pending
- submitted waiting
- interview
- rejected
- offer

- Candidate can approve/decline authorization requests. Candidate cannot otherwise edit firm statuses.
- Recruiter (Mason Kalfus) can add firms to a candidate, request authorization, and update statuses.
- Calendar/appointments: create/update/cancel with reminders/alerts.
- Prominent “tap to call / tap to email Mason” on candidate home screen.
- Add “Delete my account/data” flow (required for App Store if accounts exist).
- Import /mnt/data/firm-list-2026.md into the backend as the canonical Firms dataset.

STACK (use exactly this unless impossible)

- Mobile: Expo (React Native) + TypeScript
- Web admin: Next.js + TypeScript
- Backend: Firebase Auth + Firestore + Firebase Cloud Functions (only if needed)
- Notifications: Expo Notifications (or FCM/APNs via Firebase where appropriate)
- UI approach: simple, clean, minimal; use React Navigation on mobile.

DELIVERABLES

A) Working code for mobile app + admin web app

B) Firestore data model + security rules enforcing RBAC

C) Seed script to import firm-list-2026.md into Firestore

D) Setup docs: how to run locally, env vars, how to deploy, how to build with EAS for App Store

E) App Store readiness checklist: privacy policy placeholders, data deletion path, permissions rationale

WORK METHOD (IMPORTANT)

- Work in small, reviewable steps. After each step, summarize what changed and what to test.
- Prefer correctness and clear structure over fancy UI.
- When you need decisions, choose sensible defaults and document them.
- Add basic error handling and loading states everywhere.

BUILD PLAN (execute in this exact order)

1) Read and summarize the four spec files. Extract requirements into a checklist.

2) Create a monorepo structure:

/apps/mobile (Expo)

/apps/admin (Next.js)

/packages/shared (types, validation, constants)

3) Define Firestore schema + TypeScript types:

- users (candidate records; include role, preferences)
- firms (seeded canonical firms)
- candidateFirmStatuses (candidateId, firmId, status, timestamps, history)
- conversations/messages (candidateId, senderRole, text, attachments, timestamps)
- appointments (candidateId, time, location/link, notes, status)
- authorizationRequests (candidateId, firmId, state)

4) Implement Firebase Auth:

- OTP-based sign-in (phone or email OTP depending on easiest reliable path)
- Create user profile on first login

5) Implement candidate mobile app screens (React Navigation tabs):

- Home (Mason contact CTA + “Message Zenith”)

- Status (firm list filtered to allowed statuses)

- Calendar (appointments list + detail)

- Profile (preferences + delete account/data)

6) Implement messaging on mobile:

- Real-time message list

- Send message

- Push notification on new message

7) Implement authorization request UX on mobile:

- Show pending authorizations and allow approve/decline

8) Implement admin web console:

- Admin login (role=admin)

- Candidate list + search

- Candidate detail: profile, firms panel (add firm from master list), request authorization, update status

- Messages panel: read/respond

- Appointments panel: create/update/cancel

9) Implement RBAC security rules:

- Candidates can read/write only their own messages, appointments, profile, and authorization responses

- Candidates can read firms and their own candidateFirmStatuses

- Admins can read/write everything relevant

10) Implement firm list import:

- Parse /mnt/data/firm-list-2026.md and seed Firestore firms collection (id, name)

11) Implement account/data deletion:

- Candidate can request deletion; deletes Firestore user doc + related subcollections, and disables auth user if possible

12) Add docs:

- README with setup

- ENV variable templates
- Firebase project setup
- EAS build + submit steps

13) Add a minimal QA checklist and ensure app runs end-to-end locally.

ACCEPTANCE TESTS (must pass)

- Candidate can onboard via OTP and set preferences.
- Candidate can message Zenith; admin sees and replies; candidate receives reply and push.
- Admin can add firm to candidate and request authorization; candidate approves; admin updates status to submitted/interview/etc; candidate Status screen updates in real time.
- Candidate cannot edit statuses except authorization approve/decline.
- Candidate calendar shows appointments created by admin; reminders/alerts work.
- Account deletion removes candidate data.

NOW START.

First: read the spec files from /mnt/data, produce a requirement checklist, then scaffold the monorepo and commit the baseline structure.

Worked for 3m 55s

Plan

Zenith Legal MVP Plan (Expo + Next.js + Firebase)

Summary

We will build a monorepo MVP with a candidate mobile app, admin web console, Firebase backend, real-time messaging, push notifications, RBAC, and firm master-data import.

No code is changed yet (plan mode only).

235. I will use the spec files found in this repo root (the /mnt/data path is not mounted in this environment).

236.

237.Implementation Sequence (Exact Order)

238. Read/spec checklist: Create one requirements checklist doc from email-app-requirements.md, zl-app-transcropt.md, firm-list-2026.md, PROJECT_LOG.md, and mark each item as MVP, Later, or Out.

239. Monorepo scaffold: Create /apps/mobile (Expo + TS + React Navigation), /apps/admin (Next.js + TS), /packages/shared (types/constants/zod validation), plus root workspaces config and shared scripts.

240. Firestore schema/types: Define collections and strict TypeScript/Zod models for users, firms, candidateFirmStatuses, conversations/messages, appointments, authorizationRequests, deletionRequests (for audit), plus Firebase indexes file.

241. Auth setup: Implement candidate auth with two passwordless options (phone OTP or email link OTP-style flow), where either verified method is allowed; collect name/email/mobile and create profile on first login. Implement admin Google sign-in with allowlist and admin role claim.

242. Mobile core screens: Build React Navigation tabs Home, Status, Calendar, Profile with loading/error states and empty states.

243. Messaging: Build per-candidate thread with Firestore realtime listeners, send message, Firebase Storage file upload (any type, max 25MB), and push notifications to candidate on new admin message.

244. Authorization UX: Show pending authorization requests in mobile app and allow candidate approve/decline only.

245. Admin console: Implement admin login, candidate list/search, candidate detail page with profile, firms panel (add from canonical list), request authorization, status updates, messages panel, and appointments panel.

246. RBAC rules: Add Firestore/Storage security rules so candidates can only access their own records and only allowed writes; admins can manage all relevant records.

247. Firm import: Add seed script to parse firm-list-2026.md and import firms into Firestore; skip the Mc row by rule.
248. Account/data deletion: Add in-app “Delete my account/data” flow with hard delete of candidate-owned records and auth user removal via Cloud Function, plus deletion audit entry.
249. Docs/setup: Add README, .env.example, Firebase setup docs, local emulator docs, EAS build/submit docs, and App Store readiness checklist (privacy policy placeholder + permissions rationale + deletion path).
250. QA/end-to-end: Run emulator-based and local manual acceptance tests across mobile + admin and document pass/fail checklist.
251. Public APIs / Interfaces / Types
252. UserRole = 'candidate' | 'admin'.
253. CandidateFirmStatus = 'authorization_pending' | 'submitted_waiting' | 'interview' | 'rejected' | 'offer'.
254. AuthorizationState = 'pending' | 'approved' | 'declined'.
255. AppointmentStatus = 'scheduled' | 'canceled' | 'completed'.
256. users/{uid} fields include profile, role, preferences, verification flags, and push token.
257. candidateFirmStatuses/{id} fields include candidateId, firmId, status, updatedBy, history[].
258. conversations/{candidateId} + messages/{messageId} with sender role, text, attachment metadata, timestamp.
259. appointments/{id} includes candidateId, creator role, schedule metadata, status, reminder flags.
260. authorizationRequests/{id} includes candidateId, firmId, state, and timestamps.
261. Cloud Functions endpoints/triggers: sendPushOnMessageCreate, sendPushOnAppointmentChange, deleteCandidateAccountData.
262. Test Cases and Scenarios
263. Candidate can sign in with phone OTP and with email-link OTP flow; first login creates profile.
264. Candidate sets preferred cities and practice area and can update later.
265. Admin can add firm to candidate from seeded firm list and create authorization request.
266. Candidate can approve/decline authorization request; cannot directly edit firm status.

267. Admin can update firm status; candidate status screen updates in realtime and only shows allowed statuses.
268. Candidate and admin can exchange messages; file attachment upload works up to 25MB; candidate receives push notification.
269. Admin and candidate can create/update/cancel appointments; both see realtime updates; candidate receives reminder/alert.
270. Candidate can only read/write own permitted records (rules test with Firebase Emulator).
271. Admin allowlist enforcement blocks non-admin users from admin console data.
272. Candidate deletion flow removes Auth user and related Firestore/Storage data and leaves audit trace.
273. Assumptions and Defaults
274. Workspace source files in repo root are treated as canonical because /mnt/data is unavailable here.
275. Package manager default will be npm workspaces to keep local setup simple.
276. Admin auth uses Google sign-in + email allowlist + admin custom claim.
277. Candidate auth allows either verified phone or verified email-link method.
278. Messaging supports attachments of any file type up to 25MB (no malware scan in MVP).
279. Push notifications are candidate-only for MVP; admin uses realtime web updates.
280. Appointment edits are allowed for both admin and candidate per your choice.
281. Seed importer skips Mc and logs it as skipped for traceability.
282. After each implementation step, we will append bullet updates to project .md logs/plans (PROJECT_LOG.md, requirements checklist doc, and implementation plan doc) with “what changed” and “what to test”.
283. Implement plan
284.
285.
286. Worked for 16m 32s
287. Implemented. I built the MVP baseline across mobile, admin, shared package, Firebase backend, seed tooling, docs, and QA artifacts.
288.

289. Step-by-step log (changes + what to test)

290.

291. Full 13-step summary is here: implementation_step_log.md

292. Requirements checklist is here: requirements_checklist.md

293. What I added

294.

295. Monorepo + workspaces

296. package.json, tsconfig.base.json, .gitignore

297.

298. Shared types/constants/schemas

299. packages/shared/src/types/domain.ts, packages/shared/src/constants/options.ts, packages/shared/src/schemas/index.ts

300.

301. Candidate mobile app (Expo + TS)

302. apps/mobile/App.tsx, apps/mobile/src/navigation/RootNavigator.tsx, apps/mobile/src/screens/AuthScreen.tsx, apps/mobile/src/screens/StatusScreen.tsx, apps/mobile/src/screens/MessagesScreen.tsx, apps/mobile/src/screens/ProfileScreen.tsx

303.

304. Admin web console (Next.js + TS)

305. apps/admin/app/auth/page.tsx, apps/admin/app/dashboard/page.tsx, apps/admin/app/dashboard/candidates/[candidateId]/page.tsx

306.

307.Firebase config + RBAC rules + indexes

308. firebase.json, firestore.rules, storage.rules, firestore.indexes.json

309.

310. Cloud Functions (push, deletion, admin role helper)

311. functions/src/index.ts, functions/src/triggers/messagePush.ts, functions/src/triggers/appointmentPush.ts, functions/src/callable/deleteAccount.ts, functions/src/callable/adminRole.ts

312.

313. Firm seed import (skips Mc)

314. functions/src/scripts/seedFirms.ts

315.

316. Setup/deploy/compliance docs

317.README.md, .env.example, docs/firebase-setup.md, docs/eas-build.md, docs/app-store-readiness.md

318.

319. QA docs

320. qa/acceptance-checklist.md, qa/test-run-2026-02-25.md

321.

322. Project markdown logs updated

323. PROJECT_LOG.md, general_project_plan.md

324.

325. Verification I ran

326.

327.npm run typecheck -> pass

328. npm run build --workspace @zenith/shared -> pass

329. npm run build --workspace @zenith/functions -> pass

330. npm run build --workspace @zenith/admin -> pass

331. Mobile runtime + emulator end-to-end tests are documented but not executed yet (needs your Firebase env/project).

332.

333. Next options

334.

335. I can run Firebase emulators and execute the full acceptance checklist with you.

336. I can set up a first real Firebase project config and wire admin claim bootstrap end-to-end.

337.I can prepare a first TestFlight-ready EAS build profile using your real bundle identifiers and secrets.

338.

339. -------

340. Continue running the plan we developed

Worked for 25m 24s

Implemented next phase of the plan: local verification tooling for RBAC + emulator-ready app config.

What I changed

- Added Firebase emulator mode to mobile client:
  - firebase.ts
  - app.json
- Added Firebase emulator mode to admin client:
  - firebase.ts
- Added RBAC smoke test script (candidate vs admin permissions):
  - rbac-smoke.mjs
- Added npm scripts for security checks:
  - package.json
  - `test:rules:local`
  - `test:rules`
  - `qa:smoke`
- Updated setup/docs/env/checklists:
  - .env.example
  - README.md
  - firebase-setup.md
  - acceptance-checklist.md
- Added required markdown log bullets:
  - PROJECT_LOG.md
  - implementation_step_log.md
  - general_project_plan.md
  - test-run-2026-02-25.md

Validation run

- `npm run typecheck` -> pass
- `npm run build` -> pass
- `npm run test:rules:local` -> expected block message (requires emulators)
- `npm run test:rules` -> blocked here because `firebase` CLI is not installed in this environment

Commit

- `8454384` — “Add emulator wiring and RBAC smoke test harness”

What to test next

1. Install Firebase CLI (`firebase-tools`).
2. Run `npm run test:rules`.
3. Start apps + emulators and run full checklist in acceptance-checklist.md.

+97-189

```
.env.example
README.md
apps/admin/src/lib/firebase.ts
apps/mobile/app.json
apps/mobile/src/lib/firebase.ts
docs/firebase-setup.md
package.json
qa/acceptance-checklist.md
qa/rules/firestore.rules.test.ts
qa/rules/rbac-smoke.mjs
```

341. "What to test next
342.
343. Install Firebase CLI (firebase-tools).

344. Run npm run test:rules.

345. Start apps + emulators and run full checklist in acceptance-checklist.md."

346.

347. What's next? Plan. Then run and ask questions along the way if needed.

Worked for 17s

Execution Plan: Finish Validation and End-to-End MVP Signoff

Summary

Goal: execute the remaining validation path for the existing Zenith Legal MVP build, then close open acceptance items with evidence.

Scope now is run/verify/log, not new feature work (unless tests fail and require targeted fixes).

Chosen defaults:

- Firebase CLI install: global npm install
- Acceptance strategy: both local emulators first, then real Firebase project

Phase 0: Preconditions and Guardrails

1. Verify repo state and branch:
    - Confirm `main` is ahead by 1 commit and workspace is clean or intentionally dirty.
    - Do not rewrite history; use new commits only.
2. Install required tooling:
    - `npm install -g firebase-tools`
    - Verify: `firebase --version`
    - If global install fails, fallback to Homebrew (`brew install firebase-cli`) and re-verify.
3. Confirm local env exists:

- Ensure `.env` is present (from `.env.example`) and includes emulator toggles + Firebase keys.
- Required toggles for emulator run:
  - `EXPO_PUBLIC_USE_FIREBASE_EMULATORS=true`
  - `NEXT_PUBLIC_USE_FIREBASE_EMULATORS=true`
  - emulator host values set

4. Logging requirement for every step:
   - Append bullet to:
     - PROJECT_LOG.md
     - implementation_step_log.md
     - test-run-2026-02-25.md (or create next dated run file if needed)
   - Each bullet must include: what changed, what command/test ran, result.

Phase 1: Emulator-First Verification

1. Install project deps:
   - `npm install`
   - Verify no missing workspace deps.
2. Static checks:
   - `npm run typecheck`
   - `npm run build`
   - Record pass/fail output summary in QA run log.
3. Start emulator suite:
   - `firebase emulators:start` (auth + firestore at minimum; include functions if messaging/delete flows are tested)
   - In separate shell: `npm run test:rules`
4. RBAC smoke expectations (rbac-smoke.mjs):
   - Candidate reads own profile, not others.
   - Candidate can update own preferences.
   - Candidate cannot elevate role.

- Candidate can read own status but cannot alter status enum directly.
- Candidate can approve own auth request only.
- Admin can write firms and status records.
- If fail: capture failing assertion, map to `firestore.rules`, patch rules, rerun.

5. Local app startup checks:
    - Admin: `npm run dev:admin`
    - Mobile: `npm run dev:mobile`
    - Confirm both boot with emulator mode enabled and no Firebase init/runtime crash.

Phase 2: Full Acceptance Run on Emulators

Execute checklist in acceptance-checklist.md and mark each item pass/fail with notes.

1. Candidate onboarding:
    - Phone OTP path
    - Email link path
    - First-login user doc creation
    - Preferences save
2. Messaging:
    - Candidate sends
    - Admin reads/replies
    - Candidate receives message update
    - Attachment upload up to 25MB
    - Push behavior on emulator: verify trigger invocation/log (device push may be limited locally)
3. Firm workflow:
    - Admin add firm from canonical list
    - Request authorization
    - Candidate approve/decline
    - Admin status updates
    - Candidate realtime status update

- Candidate cannot directly edit status

4. Appointments:
    - Admin create
    - Candidate view
    - Candidate create/cancel/update
    - Realtime sync
    - Reminder/update behavior (or trigger/log-level validation locally)
5. Deletion:
    - Candidate triggers delete
    - Firestore linked docs removed
    - Auth user removed
    - `deletionRequests` audit record updated

Phase 3: Real Firebase Project Verification

1. Switch env from emulator mode to real project mode:
    - `EXPO_PUBLIC_USE_FIREBASE_EMULATORS=false`
    - `NEXT_PUBLIC_USE_FIREBASE_EMULATORS=false`
    - Ensure all production Firebase keys present.
2. Deploy backend artifacts:
    - `firebase deploy --only firestore,storage,functions`
    - Seed firms:
        - `npm run seed:firms`
    - Verify seed result count and confirm `Mc` skipped in import log.
3. Admin bootstrap:
    - Set Mason/admin claim via callable (`setAdminRoleByEmail`) from super-admin account.
    - Verify admin login allowlist + role claim enforcement.
4. Re-run high-risk acceptance subset on real backend:
    - Auth onboarding (one candidate)
    - Messaging send/reply (with attachment)

- Status + authorization flow end-to-end
- Appointment create/cancel
- Delete-account flow end-to-end

Phase 4: App Store Readiness Closure

1. Complete docs review:
   - README.md
   - firebase-setup.md
   - eas-build.md
   - app-store-readiness.md
2. Verify required App Store artifacts exist or placeholders are explicit:
   - Privacy policy URL placeholder
   - Support URL placeholder
   - Data deletion path description
   - Notification permission rationale
3. Produce final QA evidence section in QA run log:
   - Commands run
   - Pass/fail matrix
   - Known limitations
   - Remaining blockers (if any)

Failure Handling Rules

1. If a check fails, stop the sequence at that step.
2. Create minimal targeted fix only for blocker.
3. Re-run failed step + dependent checks.
4. Log root cause and resolution in project markdown logs.

Important Changes/Additions to Public APIs/Interfaces/Types

- No planned new interface/type additions for this phase by default.
- Allowed only if required by a discovered defect:
    - Firestore rules adjustments (`firestore.rules`)
    - Minor client config/env surface updates
    - Function trigger/callable bugfixes
- Any interface change must be documented in:
    - implementation_step_log.md
    - PROJECT_LOG.md
    - QA run file

Test Cases and Scenarios

1. `npm run typecheck` passes.
2. `npm run build` passes.
3. `npm run test:rules` passes under emulators.
4. Full acceptance checklist passes under emulators.
5. Real project subset acceptance passes after deploy/seed.
6. No candidate can access other candidate data.
7. Candidate cannot edit firm status directly (except authorization response).
8. Delete-account fully removes candidate data + auth user + audit trail.

Assumptions and Defaults

1. Current codebase baseline is the latest committed state (`main` ahead by one local commit).
2. Firebase CLI global install is permitted on this machine.
3. Manual device actions are available for mobile OTP and push behavior checks.
4. If emulator push cannot fully validate device delivery, function trigger logs are accepted for local pass; real-project pass must include actual delivery verification.
5. All step outcomes will be logged in project markdown files as required.

Implement plan

Worked for 6m 57s

Implemented and executed as far as environment allows.

Done now

1. Phase 0 + Phase 1 run completed.
2. Logged all changes/results in project `.md` files.
3. Commit created: `baf1a91` (`Run validation pass and log blockers`).

What passed

- `firebase --version` -> `15.7.0` (installed via Homebrew fallback)
- `java -version` -> OpenJDK installed
- `.env` created from template with emulator toggles enabled
- `npm install` -> pass
- `npm run typecheck` -> pass
- `npm run build` -> pass
- Admin startup (`npm run dev:admin`) -> pass
- Mobile startup in offline mode (`EXPO_OFFLINE=1 npx expo start --offline`) -> pass

What blocked

1. Emulator RBAC run:
    - `npm run test:rules` fails because Firestore emulator jar cannot download from: `https://storage.googleapis.com/firebase-preview-drop/emulator/cloud-firestore-emulator-v1.20.2.jar`
2. Real Firebase phase:
    - CLI not authenticated: `firebase login` not done
    - `.firebaserc` still points to placeholder project id

Logs updated

- PROJECT_LOG.md

- implementation_step_log.md
- qa/test-run-2026-02-25.md
- qa/acceptance-checklist.md
- README.md
- docs/firebase-setup.md
- general_project_plan.md

Need your input to continue

1. Can you run `firebase login` on this machine and confirm when done?
2. What is your real Firebase project id to set in `.firebaserc` and use for Phase 3 deploy/seed?

+12-0

README.md

docs/firebase-setup.md

348. "Need your input to continue
349. Can you run firebase login on this machine and confirm when done?
350. What is your real Firebase project id to set in .firebaserc and use for Phase 3 deploy/seed?"
351.
352. What do I do?

Do these 4 steps in your Terminal:

1. Login to Firebase
353. `firebase login`

- A browser opens.
- Sign in with the Google account that owns your Firebase project.
- Return to terminal and wait for success message.

2. Get your project ID

354. `firebase projects:list`

- Copy the exact `Project ID` (looks like `my-app-12345`).

3. Set it for this repo

355. `cd /Users/ryankalfus/Downloads/zenith_legal_app`

356. `firebase use <YOUR_PROJECT_ID>`

4. Confirm active project

357. `firebase use`

- It should show your project as active.

Then send me:

- “login done”
- your project ID

After that I’ll continue Phase 3 (deploy + seed + verification).

358.